

תוכנית ארכיטקטורה ופיתוח מקיפה: משחק

KFChess

מסמך זה מהווה את בסיס הפיתוח הארכיטקטוני, האסטרטגיה ההנדסית ומפת הדרכים האיטרטיבית עבור משחק השחמט **KFChess (Kung Fu Chess)** שפותח ב-Python ב-VS Code. התוכנית תוכננה בקפידה כדי לענות על דרישות הקוד הנקי, עקרונות הארכיטקטורה המתקדמים, הכנה לשינויים עתידיים (מבנה בינארי ומנוע חוקים דינמי), והגעה ל-100% כיסוי בדיקות (Test Coverage) ללא שימוש ב-Monkey Patching.

הצהרת עקרון העבודה עם LLM: אנו עושות שימוש מבוקר ומושכל ב-LLM ככלי עזר בלבד. האחריות המלאה על נכונות הקוד, הארכיטקטורה, הבדיקות והתוצרים הסופיים חלה עלינו בלבד. כל שורת קוד מובנת, מתוכננת מראש ומנומקת.

1. מענה ארכיטקטוני לדרישות הבוס (Future-Proofing)

א. מענה לעדכון 1: מוכנות למעבר לייצוג בינארי (Binary Board Representation)

כדי לאפשר מעבר עתידי לייצוג בינארי (כגון Bitboards או מערכים דחוסים) מבלי לשנות את הלוגיקה העסקית, ניעזר ב-**Encapsulation** (כמוסה) ונחלט ובדפוס העיצוב **Adapter / Interface** קוד לא יגש ישירות למבנה הנתונים הפנימי של הלוח.

- נגדיר פרוטוקול/ממשק מופשט בשם `BoardRepresentation` המגדיר כיצד מתשאלים את הלוח (למשל: `get_piece_at(position)`, `set_piece_at(position, piece)`).
- בשלב הנוכחי, נממש מחלקה בשם `TextualBoard` המממשת ממשק זה.
- כאשר נרצה לעבור לייצוג בינארי, נכתוב מחלקה חדשה בשם `BinaryBoard` שתממש את אותו הממשק בדיוק. מחלקות הלוגיקה והכלים לא יושפעו כלל מהשינוי (Open/Closed Principle).

ב. מענה לעדכון 2: מנוע חוקים דינמי והגדרת כלים על ידי המשתמש

כדי למנוע מצב של Hard-Coded של חוקי התנועה ולאפשר למשתמש לעצב משחק מותאם אישית (כמו "שחמט קונג פו בסגנון שלומי"), נשתמש בדפוס העיצוב **Strategy Pattern** (שילוב עמלא הרכבה) **Composition** (ירושה) **Inheritance** קשיחה.

- כלי (`Piece`) לא יחזיק את הלוגיקה של התנועה בתוך מחלקת יורש קשיחה (כמו `class Pawn(Piece)`), אלא יחזיק אוסף של אובייקטי תנועה מופשטים (`MovementStrategy`).
- דוגמאות לאסטרטגיות תנועה: `LinearMovement(direction_vector, max_steps)`, `JumpMovement(vectors)`.

- התנהגויות מיוחדות (כגון חייל שהגיע לשורה האחרונה ומתהפך כיוון) ימומשו כ- `TriggeredAction` או `State` `Modifier` המוזרקים לכלי כחלק מהקונפיגורציה שלו. המשתמש יוכל להגדיר קובץ JSON או YAML שיטען את הכלים הללו באופן דינמי.

ג. מענה לעדכון 3: שמירה על Code Smells ועקרונות Clean Code

העיקרון	הגדרה והנחיות בקוד	יישום מעשי בפרויקט KFCheess
DRY (Don't Repeat Yourself)	כל פיסת לוגיקה ממומשת במקום אחד בלבד.	חישוב מסלולים ישרים/אלכסוניים מרוכז בפונקציות עזר גנריות ולא משוכפל בין הרץ, הצריח והמלכה.
SRP (Single Responsibility)	כל פונקציה ומחלקה עושה דבר אחד בלבד.	מחלקת הלוח רק מנהלת את מיקום הכלים. מחלקת חוקים בודקת חוקיות. מחלקת רנדור מציגה את (RuleEngine) התוצאה.
No Hard-coded Constants	כל הערכים הקבועים והמחרוזות יושבים בקונפיגורציה.	יצירת קובץ <code>config.py</code> המכיל קבועים כמו גודל הלוח (8x8), שמות כלים, וזמני השהיה של קונג-פו שחמט.
Encapsulation (כמוסה)	אי-חשיפת פרטי מימוש פנימיים.	שימוש במשתנים פרטיים (כגון <code>self._matrix</code>). מחלקות חיצוניות משתמשות רק במתודות ציבוריות ולא ניגשות למפתחות פנימיים.

ד. מענה לעדכון 4: בדיקות עצמאיות ו-100% Test Coverage ללא Monkey Patching

- (הזרקת תלויות) **Dependency Injection** במקום להשתמש ב-Monkey Patching שמשנה את הקוד בזמן ריצה באופן לא מוסרי, נזריק תלויות. אם פונקציה צריכה לקרוא את הזמן הנוכחי או לקבל קלט, נזריק לה אובייקט `Clock` או `InputProvider`. בזמן הבדיקות, נזריק Mock Object פשוט הממומש על ידינו.
- **כלי הבדיקה:** נשתמש ב-`pytest` ובתוסף `pytest-cov` להפקת דוח HTML מקיף שמראה בדיוק אילו שורות כוסו. שאיפת היעד היא 100%.
- **הערת גיט:** בתחילת קובץ ה-`main.py`, תופיע הערה ברורה עם ה-URL של ה-Git Repository לטובת ה-Reviewer.

2. מפת הדרכים לפיתוח איטרטיבי (לפי שלבי הבוס)

Iteration 1

שלב 1: Board + Pieces Parsing

בניית **תיאור** הבסיסית של הלוח וטעינת/ניתוח מצב הלוח והכלים מתוך קלט (מחרוזת או קובץ קונפיגורציה).

- מימוש מחלקת הלוח (`Board`) והכלים (`Piece`) תחת עקרון ה-Encapsulation והכנה לייצוג בינארי.
- כתיבת Parser שיועד לקחת ייצוג טקסטואלי (למשל FEN או פורמט מותאם אישית) ולהקים לפיו את האובייקטים בזיכרון.

Iteration 2

שלב 2: Click Mapping + Controller Selection

תיאור: מימוש שכבת השליטה (Controller) ותרגום לחיצות עכבר/קלט של משתמש למיקומים על גבי הלוח.

- יצירת Controller המפריד בין ה-UI לבין ה-Model (ארכיטקטורת MVC או MVP).
- תרגום קואורדינטות מסך או פקודות לקואורדינטות לוח (למשל, לחיצה על פיקסל X,Y מתורגמת למשבצת `e4`).

Iteration 3

שלב 3: Basic Piece Movement Rules

תיאור: החלת חוקי התנועה הבסיסיים של הכלים ללא התחשבות בחסימות או כלים אחרים על הלוח.

- מימוש ה-`MovementStrategy` עבור סוגי התנועות השונים (קווים ישרים לצריח, אלכסונים לרץ, קפיצות לפרש).
- בדיקה שהכלים מייצרים את רשימת המשבצות הפוטנציאליות הנכונה על לוח ריק.

Iteration 4

שלב 4: Blocking + Simple Capture Rules

תיאור: הוספת חוקי חסימה (כלי לא יכול לעבור דרך כלי אחר, פרש כן) וחוקי הכאה בסיסיים.

- עדכון מנוע החוקים כך שמסלול התנועה נקטע כאשר הוא פוגש כלי אחר.
- הבחנה בין כלי עמית (חסימת תנועה) לכלי יריב (אפשרות הכאה ועצירה).

Iteration 5

שלב 5: Pawns

תיאור: מימוש התנהגות כלי החייל (Pawn) בשל מורכבותו וחוקיו הייחודיים.

- תנועה קדימה (צעד אחד או שניים בצעד ראשון) לעומת הכאה באלכסון בלבד.

- הכנה ארכיטקטונית לחוקי הכתרה (Promotion) והכאה דרך הילוך (En Passant) באמצעות אסטרטגיות מותנות.

Iteration 6

שלב 6: Real-time Movement / Wait / Arrival (Kung Fu Chess)

- **תיאור:** הפיכת המשחק למשחק בזמן אמת (שחמט קונג פו) - ללא תורות, עם זמני טעינה (Cooldown) לכל כלי.
- הכנסת פונקציונליות של זמן: לכל כלי יש מצב (Ready או Cooldown). לאחר תנועה, הכלי נכנס להשהיה.
- שימוש ב-Dependency Injection של שעון מערכת על מנת לאפשר בדיקות Unit Tests אמינות של רכיב הזמן האמיתי ללא שימוש ב-`time.sleep` אמיתי בטסטים.

3. ארכיטקטורת קוד בסיסית ושלד הפרויקט (Boilerplate)

להלן מבנה הקבצים המוצע ב-VS Code לשמירה על סדר וארגון:

```
KFChess/
├── .gitignore
├── README.md
├── requirements.txt
├── config/
│   └── game_config.py           # קבועים, הגדרות לוח, זמני Cooldown
├── src/
│   ├── __init__.py
│   ├── main.py                 # בסקריפט Git של ה-URL-נקודת כניסה, מכיל את ה
│   ├── board.py                # ניהול הלוח והממשק המופשט
│   ├── pieces.py               # ישויות הכלים ואסטרטגיות התנועה
│   ├── engine.py               # מנוע חוקים ושליטה בזמן אמת
│   └── controller.py           # ניהול הקלט ותרגום לחיצות
└── tests/
    ├── __init__.py
    ├── test_board.py
    ├── test_pieces.py
    └── test_engine.py
```

4. קוד שלד בסיסי ומודל הזרקת תלויות (src/main.py)

דוגמה לשלד ראשוני המציג קוד נקי, ללא ערכים קשיחים, עם הזרקת תלויות מלאה לטובת בדיקות איכותיות:

```
# Git Repository URL: https://github.com/your-team/KFChess.git
# Author: Development Team
```

```

# Clean Code Checked: DRY, SRP, Encapsulation, No Hard-coded constants

from config.game_config import BOARD_SIZE, DEFAULT_COOLDOWN

class IBoardRepresentation:
    """ממשק מופשט לייצוג הלוח - מאפשר מעבר עתידי לבינארי בקלות"""
    def get_piece_at(self, row: int, col: int):
        raise NotImplementedError

    def set_piece_at(self, row: int, col: int, piece):
        raise NotImplementedError

class TextualBoard(IBoardRepresentation):
    """מימוש טקסטואלי/מטריציוני ראשוני של הלוח"""
    def __init__(self):
        self._matrix = [[None for _ in range(BOARD_SIZE)] for _ in range(BOARD_SIZE)]

    def get_piece_at(self, row: int, col: int):
        if 0 <= row < BOARD_SIZE and 0 <= col < BOARD_SIZE:
            return self._matrix[row][col]
        return None

    def set_piece_at(self, row: int, col: int, piece):
        if 0 <= row < BOARD_SIZE and 0 <= col < BOARD_SIZE:
            self._matrix[row][col] = piece

class MovementStrategy:
    """אסטרטגיית תנועה מופשטת - מאפשרת למשתמש להגדיר כלים כרצונו"""
    def calculate_moves(self, row: int, col: int, board: IBoardRepresentation):
        raise NotImplementedError

class LinearMovement(MovementStrategy):
    def __init__(self, d_row: int, d_col: int, max_steps: int):
        self.d_row = d_row
        self.d_col = d_col
        self.max_steps = max_steps

    def calculate_moves(self, row: int, col: int, board: IBoardRepresentation):
        valid_moves = []
        # לוגיקת חישוב צעדים גנרית לפי וקטור
        return valid_moves

class Piece:
    """(Composition) מורכבת מאסטרטגיות - מיישמת כלי גנרית"""
    def __init__(self, name: str, color: str, movement_strategies: list):
        self.name = name
        self.color = color
        self.strategies = movement_strategies
        self._cooldown_remaining = 0.0

    def get_available_moves(self, row: int, col: int, board: IBoardRepresentation):
        moves = []
        for strategy in self.strategies:
            moves.extend(strategy.calculate_moves(row, col, board))
        return moves

def main():

```

```
print("Welcome to KFChess - Production Base Ready.")

if __name__ == "__main__":
    main()
```

5. מדריך פקודות עבודה והפקה של 100% Test Coverage

כדי להבטיח את איכות הקוד ולוודא עמידה ביעדי הבדיקות ללא ביצוע שינויים בזמן ריצה, יש להשתמש בפקודות הבאות בטרמינל של VS Code:

התקנת הספריות הנדרשות:

```
pip install pytest pytest-cov
```

הרצת הבדיקות והפקת דוח HTML לכיסוי קוד:

```
pytest --cov=src --cov-report=html tests/
```

פקודה זו תיצור תיקייה בשם `htmlcov/` פתיחת הקובץ `index.html` בתוך הדפדפן תציג בצורה ויזואלית ומדויקת אילו שורות קוד לא כוסו בבדיקות, ותאפשר לנו להשלים את הבדיקות עד ל-100% מלאים.